Inheritance: Introduction

- **Vehicle –** has wheels (diff num), has maximum speed, manufacturer etc..
  - **Automobile –** has engine, has gears etc...
    - **PassangeVehicle –** passanger capacity etc...
    - **GoodCarrier –** max load in weight, max load in volume etc...
  - **Cart –** Some animal to pull the cart, number of animals required etc...

```
class Vehicle {
        int wheels_count;
        float max_speed;
        string manufacturer;
public:
        Vehicle(int, float, string);
        void print();
};
```

```
class Automobile {
        int wheels_count;
        float max_speed;
        string manufacturer;
        float engine_cc;
        int gear_count;
public:
        Automobile(int, float, string,
float, int);
        void print();
};
```

```
class Cart {
        int wheels_count;
        float max_speed;
        string manufacturer;
        string animal_type;
        int animal_count;
public:
        Cart(int, float, string,
string, int);
        void print();
};
```

- Is there duplication of code?
- Can we resuse code instead?

Inheritance: Introduction

- Inheritance (IS-A relationship)
  - Mechanism of deriving new class from existing class is called Inheritance (or derivation)
    - Existing class used in inheritance is called – base class / parent class / super class
    - Newly created class is called – derived class / child class / sub class
  - Syntax of inheritance is as follows:

```
class derived-class : visibility-mode base-class
{
        // Members of derived class
}
```

base-class

↑

derived-class

  - ***derived-class* IS-A *base-class***.
    - visibility-mode could be private / protected / public
      - Its optional, by default its private
      - Though visibility mode is often public, will look into it first

- Access specifier determine how elements of the base class are inherited by the derived class
- Access specifier is public : all the public members of the base class become public members of the derived class
- Access specifier is private : all the public members of the base class become private members of the derived class
- In any case private members are not inherited
- If access specifier is not present it is private by default if derived class is class . If derived class is struct then access spicier is public by default

Inheritance: Introduction

Data Members

Derived class inherits all data members of Base class

Derived class may add data members of its own

Member Functions

Derived class inherits all member functions of Base class

Derived class may add member functions of its own

Access Specification

Derived class cannot access private members of Base class

Derived class can access protected members of Base class

Construction-Destruction

A constructor of the Derived class must first call a constructor of the Base class to construct the Base class instance of the Derived class

The destructor of the Derived class must call the destructor of the Base class to destruct the Base class instance of the Derived class

Constructors and destructors

- Base class and derived class both have constructor and destructor
- Base class constructor is executed before the derived class constructor. The reverse is true for destructor
- In case of arguments, all necessary arguments to both the base class and derived class are passed to the derived class's constructor

```
 derivedclass constructor(ardlist): base(arglist){
body of derived class constructor}
```

constructor

- If the derived class does not need an argument, it ignores and simply passes to base class. In this case derived class constructor has one argument if base required one.
- As long as no base class constructor takes any arguments the derived class need not have a constructor. But if base class contains constructor with one or more arguments then it is mandatory for the derived class to have a constructor with one or more arguments

```
class Base{

Base()
   {
      cout<<"Constructor: Base"<<endl;
   }
   ~Base()
   {
      cout<<"Destructor: Base"<<endl;
   }
};

class Other
{

Other()
   {
      cout<<"Constructor: Other"<<endl;
   }
   ~Other()
   {
      cout<<"Destructor: Other"<<endl;
   }
};
```

```
class Derived:public Base{

   Other o;
   Derived()
   {
      cout<<"Constructor:
Derived"<<endl;
   }
   ~Derived()
   {
      cout<<"Destructor:
Derived"<<endl;
   }
};

int main()
{
   Derived derived;
   return 0;
}
```

Output ??

```
class Base{

Base()
   {
      cout<<"Constructor: Base"<<endl;
   }
   ~Base()
   {
      cout<<"Destructor: Base"<<endl;
   }
};

class Other
{

Other()
   {
      cout<<"Constructor: Other"<<endl;
   }
   ~Other()
   {
      cout<<"Destructor: Other"<<endl;
   }
};
```

```
class Derived:public Base{

   Other o;
   Derived()
   {
      cout<<"Constructor:
Derived"<<endl;
   }
   ~Derived()
   {
      cout<<"Destructor:
Derived"<<endl;
   }
};

int main()
{
   Derived derived;
   return 0;
}
```

Output

Error: Base::Base() is private

Error Other::Other() is private

```
class Base{
public:
    Base()
    {
        cout<<"Constructor: Base"<<endl;
    }
    ~Base()
    {
        cout<<"Destructor: Base"<<endl;
    }
};
class Other
{
public:
    Other()
    {
        cout<<"Constructor: Other"<<endl;
    }
    ~Other()
    {
        cout<<"Destructor: Other"<<endl;
    }
};
```

```
class Derived:public Base{
public:
    Other o;
    Derived()
    {
        cout<<"Constructor: Derived"<<endl;
    }
    ~Derived()
    {
        cout<<"Destructor: Derived"<<endl;
    }
};

int main()
{
    Derived derived;
    return 0;
}
```

Output ??

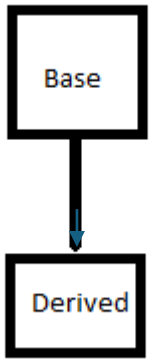
```
class Base{
public:
    Base()
    {
        cout<<"Constructor: Base"<<endl;
    }
    ~Base()
    {
        cout<<"Destructor: Base"<<endl;
    }
};
class Other
{
public:
    Other()
    {
        cout<<"Constructor: Other"<<endl;
    }
    ~Other()
    {
        cout<<"Destructor: Other"<<endl;
    }
};
```

```
class Derived:public Base{
public:
    Other o;
    Derived()
    {
        cout<<"Constructor: Derived"<<endl;
    }
    ~Derived()
    {
        cout<<"Destructor: Derived"<<endl;
    }
};

int main()
{
    Derived derived;
    return 0;
}
```

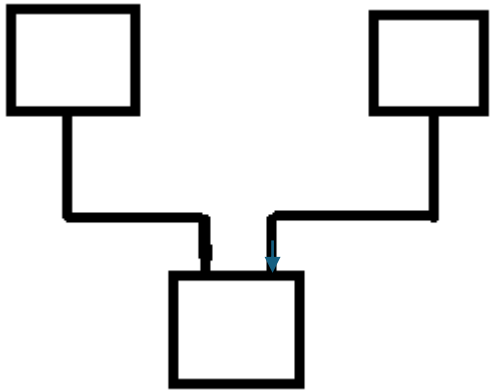
Output :
Constructor: Base
Constructor: Other
Constructor: Derived
Destructor: Derived
Destructor: Other
Destructor: Base

Types of inheritance

- Single inheritance : one base class and one derived class

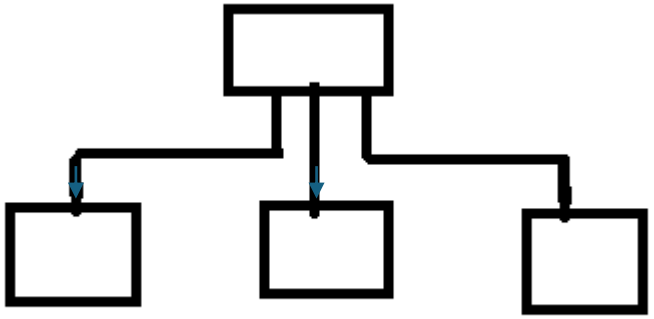


- Multiple : one with several base class

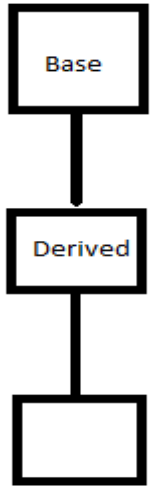


Types of inheritance

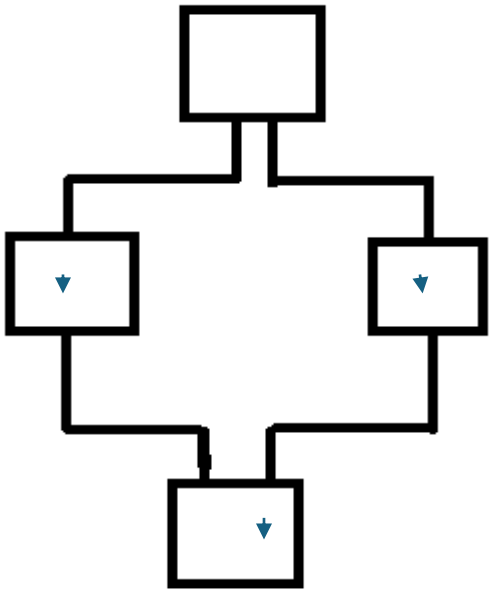
- Hierarchical inheritance: one base class may be inherited by many classes



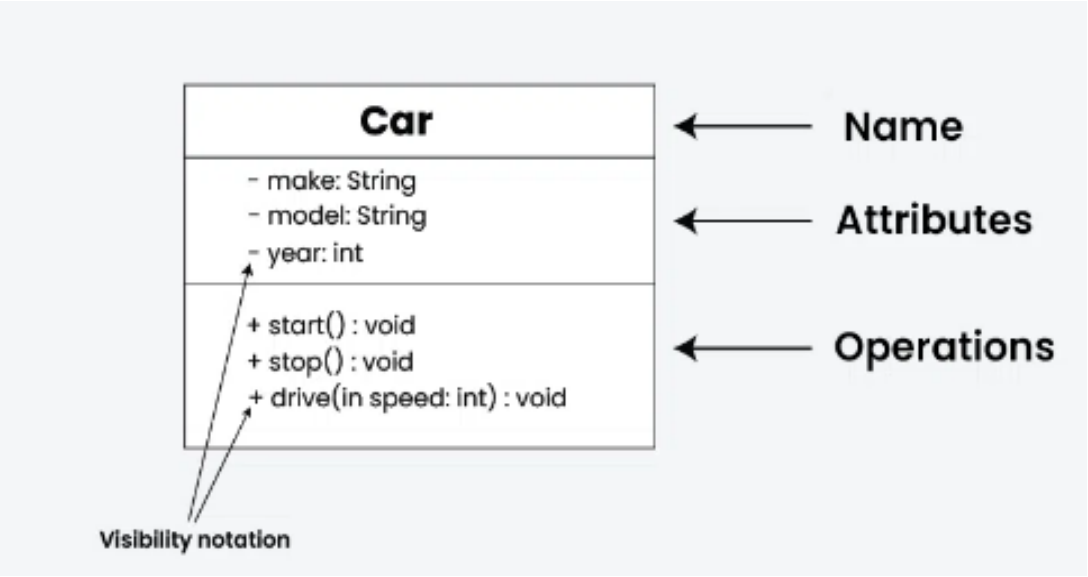
- Multilevel : mechanism of deriving a class from another derived class is known as multilevel inheritance



- Hybrid



Class diagram



- Visibility notations indicate the access level of attributes and methods. Common visibility notations include:
  - + for public (visible to all classes)
  - - for private (visible only within the class)
  - # for protected (visible to subclasses)
  - ~ for package or default visibility (visible to classes in the same package)

Single inheritance

```
class B
{
  int a;
  public:
  int b;
  void set_a();
  int get_a(void);
  void show_a(void);
  };
```

```
class D: public B
 {
  int c;
  public:
   void mul(void);
   void dispaly(void);
   };
```

```
void B::set_a(void)
{
 a=5;
 }

int B::get_a()
 {
 return a;}
 void B::show_a()
 { cout<<"a="<<a<<endl;}

 void D::mul()
 {
  c=b*get_a();
 }
 void D::dispaly()
 {
  cout<<"a="<<get_a()<<endl;   //show_a();
  cout<<"b="<<b<<endl;
  cout<<"c="<<c<<endl;
  }
```

```
int main()
{
    D d;
    d.get_a();
    d.b=10;
    d.mul();
    d.dispaly();
    d.b=20;
    d.mul();
    d.dispaly();
    return 0;
}
```

Q:- Declare a base class called employee having data members like empid and name. derive a class called salary-details having memebres like basic_slary and peks. Define necessary functions.

Main function to create an array of class salary-details to store details of employee and to print details of employee having salary more than value given by user at input

```
class employee
{
  char empid[10];
  char name[10];
  public:
  void setidname()
  {
    cout<<"enter emp name and id\n";
    cin>>empid>>name;
  }
  void display_idname()
  {
   cout<<"empid  "<<empid<<"name  " <<name<<endl;
   }
   };
```

```
class salary_details:public employee
  {
   float basic_salary;
   float perks;
   public:
   void setbasic()
   {
    cout<<"enter basic salary and perks\n";
    cin>>basic_salary>>perks;
    }
    void displaybasic()
    {
    cout<<"basic salary  "<<basic_salary<<"perks  "<<perks<<endl;
     }
     float totalsalary()
     {
            return (basic_salary+perks);
     }

    };
```

```
int main()
    {
     salary_details s[10];
     int i;
     for(i=0;i<4;i++)
     {
     s[i].setidname();
     s[i].setbasic();
     }
     float amount,t_salary;
     cout<<"enter amount\n";
     cin>>amount;
     for(i=0;i<4;i++)
     {
      t_salary= s[i].totalsalary();
      if(t_salary>amount)
      {   s[i].display_idname();
                s[i].displaybasic();
                cout<<"total  "<<t_salary<<endl;
      }
    }
    return 0;
    }
```

Protected members

- Derived class does not have access to the private members of the base class
- One way is make them public but then no advantage of data hiding
- We want to keep a member of base class private but still allow a derived class access to it. This possible through access specifier protected
- Protected member is accessible by the member functions within its class and any class immediately derived from it. It cannot be accessed by the functions outside these two classes.

Protected members

```
class class_name
{
 //private members
  protected:


Public:
};
```

when mode of inheritance is public, protected members of base class become protected in derived class

When mode of inheritance is private, a protected member of the base class becomes private member of the derived class.

When mode of inheritance is protected, public and protected member of base class become protected members of derived class. Private members are not inherited

Visibility of inherited members

Base calss visisbility	Derived class visisbility		
	Public	Private	Protected
private	Not inherited	Not inherited	Not inherited
protected	protected	private	protected
public	public	private	protected

protected

- What are the various functions that can have access to protected members?
- Member function
- A function that is friend of the class
- A member function of a class that is a friend of the class
- A member function of derived class

First two function have direct access to both private and protected data

But member function of derived class can directly access protected but not private. They can access through member functions of base class.

```
 class base
{
 protected :
int a,b;
public:
  void setab(int n,int m){ a=n;b=m;}
};
class derived: protected base
{
  int c;
  public:
    void setc(int n){ c=n;}
  void showabc()
  {
   cout << a<<“    “<<b<<“    “<<c<<endl;
}
};
```

```
int main()
  {
   derived ob;
   ob.setab(1,2);
   ob.setc(3);
   ob.showabc();
   return 0;
}
```

```
 class base
{
 protected :
int a,b;
public:
  void setab(int n,int m){ a=n;b=m;}
};
class derived: protected base
{
  int c;
  public:
    void setc(int n){ c=n;}
  void showabc()
  {
   cout << a<<“   “<<b<<“   “<<c<<endl;
}
};
```

```
int main()
 {
  derived ob;
  ob.setab(1,2);  //error
  ob.setc(3);
  ob.showabc();
  return 0;
}
```

setab() is **public** in base. But because you inherited base as **protected**, all public and protected members of base become **protected** in derived.

So we cannot call it from main() using ob.setab(...) because outside world cannot access protected members.

Multilevel inheritance

```
class student
{
 int roll_number;
 public:
 void set_number(int a){ roll_number=a;}
 void put_number(){ cout<<"rol number" <<roll_number;}
  };
```

```
class test: public student
 {
   protected:
   float sub1;
   float sub2;
   public:
   void set_marks(float,float);
   void put_marks(){cout<<"sub1"<<sub1<<"sub2"<<sub2<<endl;}
   };
   void test::set_marks(float x, float y)
   {
    sub1=x;
    sub2=y;
    }
```

```
class result:public test
 {
  float total;
  public:
  void display();
  };
  void result::display()
  {
    total=sub1+sub2;
    put_number();
    put_marks();
    cout<<"total"<<total;
    }
```

```
int main()
    {
    result student1;
      student1.set_number(111);
    student1.set_marks(75,45);
    student1.display();
    return 0;
    }
```

- Result class have

Protected sub1,sub2
Public set_number(int), put_number(), set_marks(float,float), put_marks(), display()

Multiple inheritance

```
class M
{
 protected:
 int m;
Public:
 void set_m(int a){ m=a;}
};
 class N
{
 protected:
int n;
public:
 void set_n(int b){ n=b;}
};
```

```
 class P: public M, public N
{
  public:
  void display()
{
 cout<<“m=    “<<m<<endl;
cout<<“n=    “<<n<<endl;
cout<<“m*n=    “<<m*n<<endl;
}
};
```

```
int main()
 {
  P p;
  p.set_m(10);
  p.set_n(20);
 p.display();
  return 0;
}
```

Multiple inheritance

- When multiple base classes are inherited, constructors are executed in the order left to right the way in which base classes are specified.
- Destructors are executed in opposite order
- Base class constructors arguments are specified through derived class constructor

Multiple inheritance

```
class M
{
 protected:
 int m;
public:
 M (int a){ cout<<"base 1 const\n";m=a;}
 ~M(){cout<<"dest base 1\n";}
};

 class N
{
 protected:
int n;
public:
 N(int b){cout<<"base 2 const\n"; n=b;}
 ~N(){cout<<"dest base 2\n";}
};
```

```
 class P: public M, public N
{
 public:
 P(int i,int j):M(i),N(j){ cout<<"derived const\n";}
 void display()
{
 cout<<"m=    "<<m<<endl;
cout<<"n=    "<<n<<endl;
cout<<"m*n=    "<<m*n<<endl;
}
~P(){cout<<"destructor derived\n";}
};
```

```
int main()
{
 P p(3,4);
 p.display();
 return 0;}
```

Suppose if we write

```
P(int i,int j):N(i),M(j){ cout<<"derived const\n";}
```

Then also M base class constructor invoked first because it is declare first in the derived class header line

Multilevel

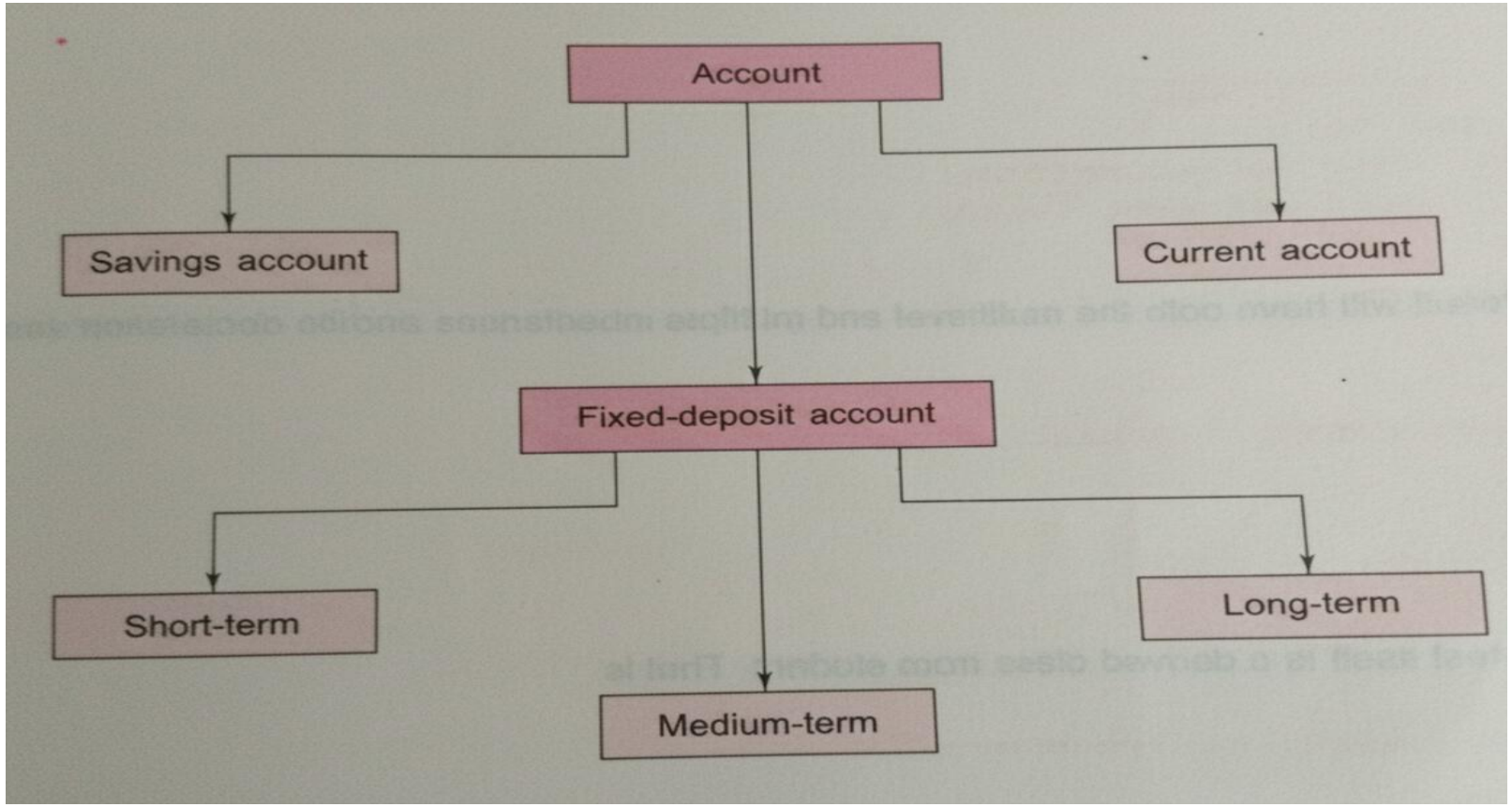
```
class B1
{
int a;
public:
B1(int x){cout<<"base 1 const\n"; a=x;}
int geta(){ return a;}
};
class D1: public B1
{
 int b;
 public:
D1(int x,int y):B1(y)
{
cout<<"base 2 const\n" ;
b=x;}
int getb(){return b;}
};
```

```
class D2:public D1
{
int c;
public:
D2(int x,int y,int z):D1(y,z)
{
cout<<"derived const\n"  ;
c=x;}
void show()
{
cout<<"a=  "<<geta()<<"   "<<"b=  "<<getb()<<"  "<<"c=
"<<c<<endl;
}
};
int main()
{
 D2 ob(1,2,3);
ob.show();
return 0;
}
```

Hierarchical

- Here features of one level are shared by many others below that level

Base class will include all the features that are common to the subclasses



Example:- base class vehicle and two derived classes car and truck

```
#include <iostream.h>
class vehicle
{
int num_wheels;
 int range;
 public:
  vehicle(int w,int r)
  {
   cout<<"base const\n";
   num_wheels=w;
   range=r;}
void showv()
{
 cout<<"wheels "<<num_wheels<<"range ="<<range<<endl;}
};
```

```
 class car: public vehicle{
  int passengers;
  public:
   car(int w, int r,int p):vehicle(w,r){cout<<"derived\n";passengers=p;}
   void show() { showv(); cout<<"passegenrs"<<passengers<<endl;}
};
 class truck: public vehicle
{
  int loadlimit;
  public:
 truck(int w,int r, int l):vehicle(w,r){ cout<<"truck const\n"; loadlimit=l;}
void show()
{
 showv();
cout<<"loadlimit  "<<loadlimit<<endl;
}
};
```

```
int main()
{
 car c(4,500,5);
truck t(12,1200,30000);
cout<<"car"<<endl;
c.show();
cout<<"truck"<<endl;
t.show();
return 0;
}
```

Example:-

```
class staff
{
 char code[10];
 char name[10];
 public:
 staff()
 {
  cout<<"enter code and name\n";
  cin>>code>>name;
  }
  void display()
  {
   cout<<"code="<<code<<"name="<<name<<endl;
  }
 };
```

```
class teacher:public staff
  {
   int n;
   char subject[5][5];
   int publication;
   int experience;
   public:
   teacher()
   {
    cout<<"enter no of subjct no of publications and experience\n";
    cin>>n>>publication>>experience;
    cout<<"eneter subject name\n";
    for(int i=0;i<n;i++)
    cin>>subject[i];
    }
```

```
void display_t()
   {
    display();
    cout<<"publication experience and subject\n";
    cout<<publication<<"  "<<experience<<endl;
    for(int i=0;i<n;i++)
     cout<<subject[i]<<endl;    }
  };
```

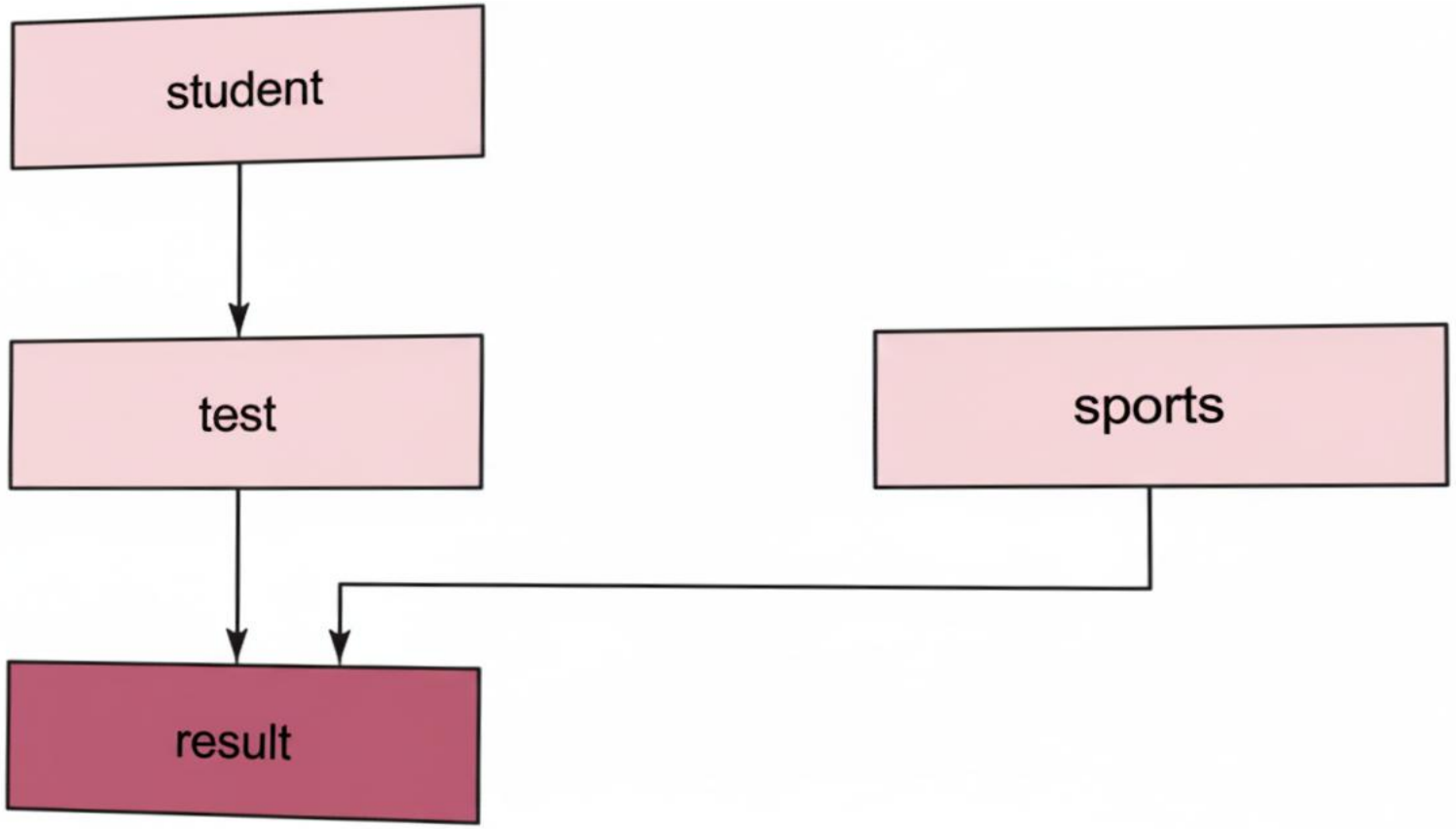
```
class officer:public staff
    {
     char grade;
     public:
     officer()
     {
      cout<<"enter grade\n";
      cin>>grade;
     }
     void display_o()
     {
            display();
            cout<<"grade="<<" "
<<grade<<endl;;
    }  };
```

```
int main()
  {
   teacher t[2];
   officer o[2];
   int i;
   for(i=0;i<2;i++)
      t[i].display_t();
    for(i=0; i<2;i++)
        o[i].display_o();
    return 0;
  }
```

H.W :- Create a base class called building that stores the number of floors a building has, the number of rooms and its total square footage. Create a derived class called house that inherits building and also stores the number of bedrooms and the number of bathrooms. Next create a derived class called office that inherits building and stores the number of fire extinguishers and the number of telephones. Write parameterized constructors for initialization. Also write necessary member functions. Crete an array of house and display all information if number of bed rooms are more than 3 otherwise generate exception. Create an array of office and display all information if they have fire extinguishers otherwise generate exception. Identify type of inheritance. Write main to demonstrate use of above functions

hybrid

- Two or more types of inheritance are used



Ambiguity

```
//single inheritance
class A
{
public:
void display(){cout<<"class A\n";}
};
class B : public A
 {
public:
void display(){cout<<"class B\n";}
};
```

```
int main()
  {
   B b;
   b.display();  //invoked display of derived class B
  b.A::display(); //invoked display of base class
   return 0;
}
//This is known as function overriding
```

Function Overriding

- **Function Overriding** happens in **inheritance** when a **derived class** provides its own implementation of a function that already exists in the **base class**, with the **same function signature** (same name, same parameters).
- In short: **Same function name + same parameters + different class (inheritance)**
- The base class function is **redefined** in the derived class.
- It allows achieving **runtime polymorphism** when used with pointers/references to the base class.
- **Key Points**
  - Must have **inheritance** (base & derived class).
  - Function **signature must match** (name + parameters).
  - Return type must be the same (except for *covariant return type* cases).
  - Use of **virtual keyword** in the base class makes overriding work in **runtime polymorphism**.

Example:

```
#include <iostream>
using namespace std;

class Animal {
public:
   void sound() {
      cout << "Animal makes a sound" << endl;
   }
};
```

```
class Dog : public Animal {
public:
   void sound() {
      cout << "Dog barks" << endl;
   }
};

int main() {
   Animal a;
   Dog d;

   a.sound();   // Animal makes a sound
   d.sound();   // Dog barks
}
```

sound() in Animal is the overridden function.
sound() in Dog is the overriding function.

Difference between Overloading and Overriding

Feature	Overloading	Overriding
Definition	Same function name, different parameters (in same class).	Same function name, same parameters (in different class, with inheritance).
Binding	Compile-time (static).	Runtime (dynamic, with virtual).
Inheritance	Not required.	Must have inheritance.
Purpose	Increases readability (same name for similar tasks).	Achieve runtime polymorphism.

Ambiguity in multiple inheritance

- Function with same name appears in more than one base class

```
class M
{
public:
void display(){cout<<“class m\n”;}
};
class N
 {
public:
void display(){cout<<“class n\n”;}
};
```

```
class p: public M,public N
  {
   public:
    void display()
   {
    M::display();
   N::display();
}
};
Int main()
{
  p d;
d.display();
}
```

Virtual base class

- Consider hybrid with multilevel, multiple and hierarchical
- The child inherits grandparent via two separate paths
- All the public and protected members of grandparent are inherited into child twice first via parent1 and again via parent 2. this introduces ambiguity when a member of grand parent is used by child and should be avoided
- Make common base class as virtual base class while declaring the direct or intermediate base classes

```
 class grandparent
{
};
  class parent1: public virtual grandparent
{
};
  class parent2: virtual public grandparent
{
};
class child: public parent1,public parent2
{
};
```

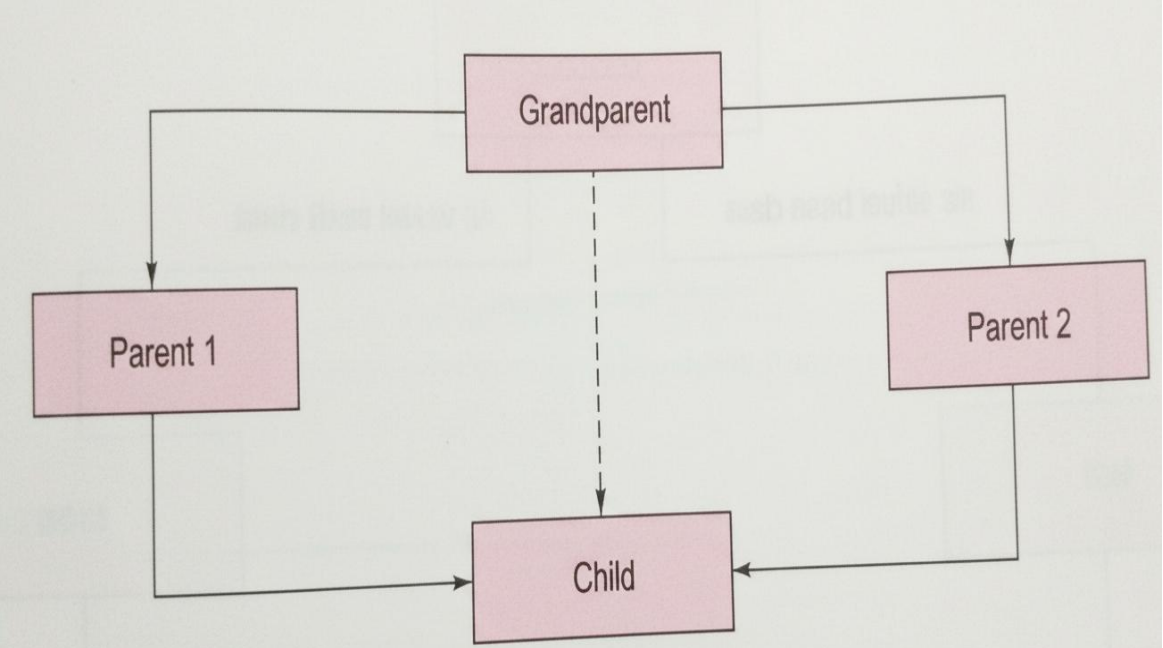


Fig. 8.12 *Multipath inheritance*

Virtual base class

- When class is made a virtual base class c++ takes necessary care to see that only one copy of that class is inherited regardless of how many inheritance paths exist between the virtual base class and derived class
- Keyword virtual and public may be used in either order
- In situation like one just described in which a derived class indirectly inherits same base class more than once, it is possible to prevent two copies of the base form being present in the derived object by having that base class inherited as virtual by any derived classes
- That prevents two or more copies of base from in any subsequent derived class that inherits base class indirectly
- Virtual key word precedes the base class access specifier when it is inherited by a derived class

Example

```
 class base
{
  public:
  int i;
};
 class derived1: virtual public base
{
  public:
  int j;
};
 class derived2: virtual public base
{
 public:
  int k;
};
class derived3: public derived1,public derived2
{
  public:
   int product()
   {
    return i*j*k;
}
};
 int main()
{
   derived3 ob;
  ob.i=10;
  ob.j=3;
  ob.k=5;
cout<<“product=“<<ob.product()<<endl; return 0;}
```

```
#include <iostream>
using namespace std;
class base
{
  public:
  int i;
};
 class derived1:  public base
{
  public:
  int j;
};
```

```
 class derived2:  public base
{
 public:
  int k;
};
class derived3: public derived1,public derived2
{
  public:
   int product()
   {
    return derived1::i*j*k;
}
};
```

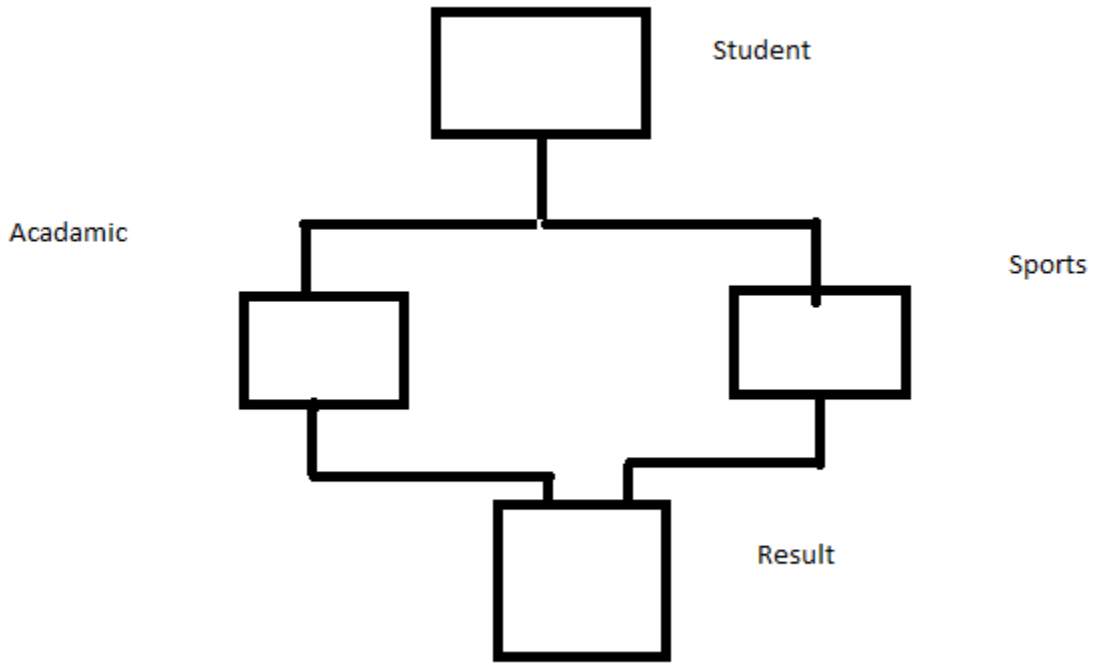
```
int main()
{
   derived3 ob;
  ob.derived1::i=10;
  ob.j=3;
  ob.k=5;
cout<<"product="<<ob.product()<<endl; return 0;}
```

discussion

- if derived1 and derived2 has not inherited base as virtual the statement ob.i=10; would have been ambiguous and compile time error would have been resulted
- Keep mind that when base class is inherited virtual by a derived class that base class still exists in both derived classes

```
derived1 ob;
ob.i=10;
```

The only difference between a normal base class and a virtual one occurs when an object inherits the base class more than once.


Student
Acadamic
Sports
Result

```
class student
 {
  char id[10];
  char name[10];
  public:
  student()
  {
   cout<<"enter id and name\n";
   cin>>id>>name;
   }
   void display()
   {
    cout<<"id="<<id<<"name="<<name<<endl;
   }
  };
```

```
class acadamic:virtual public student
  {
   int n;
   char subject[5][5];
   char grade[10];
   char credit[10];
   public:
   acadamic()
   {
    cout<<"enter no of subjct \n";
    cin>>n;
    cout<<"eneter subject name credit and grade\n";
    for(int i=0;i<n;i++)
    cin>>subject[i]>>credit[i]>>grade[i];
    }
    void display_e()
    {
     //display();
     cout<<"subject name, credit and grade\n";
                for(int i=0;i<n;i++)
      cout<<subject[i]<<"   "<<credit[i]<<" "<<grade[i]<<endl;

    }
     float cal_acadamic();    };
```

```
float acadamic ::cal_acadamic()
   {
     float sum=0,sum_c=0;
     for(int i=0;i<n;i++)
          {

            sum=sum+ (10-2*(grade[i]-'A'))*credit[i];
            sum_c=sum_c+credit[i];
          }
          return (sum/sum_c);
     }
```

```
class sports:virtual public student
{
 float points;
 public:
 sports()
 {
  cout<<"enter points\n";
  cin>>points;
 }
 float getpoint()
 {
        return points;
 }
 void display_s()
 {
        cout<<"points="<<" " <<points<<endl;;
 }
};
```

```
class result: public acadamic,public sports
{
  float spi;
  public:
  void cal_result()
  {
         spi=cal_acadamic()+getpoint();
  }
        void display_r()
        {
          display();
          display_e();
          display_s();
          cout<<"spi= "<<spi<<endl;
        }
};
```

```
int main()
  {
   result r;
   r.cal_result();
   r.display_r();
    return 0;
  }
```

```
#include<iostream>
using namespace std;
class base
{
  int i;
  public:
  base(int x=0)
  {
   cout<<"base\n";
   i=x;
   }
   void show()
   {
      cout<<i<<endl;
   }
};
class d: virtual public base
{   int j;
   public:
   d(int y){j=y;}
   void dis()
   {

      cout<<j<<endl;    }   };
class e: virtual public base
 {
    int k;
 public:
  e(int z){k=z;}
  void dis1()
  {

     cout<<k<<endl;
  }
 };
 class f: public d,public e
 {
   public:
   f(int x,int y,int z): d(y),e(z),base(x){ }
  };
  int main()
  {
   f ob(3,7,9);
   ob.show();
   ob.dis();
   ob.dis1();
  }
```

Nesting of classes

- Inheritance is mechanism of deriving certain properties of one class into another
- C++ support another way to do this is nesting. That is a class can contain objects of other classes as its members
- Nested object is created in two stages
- First the member objects are created using their respective constructors and then the other members are created. This means, constructors of all member objects should be called before the own constructor body is executed

```
class alpha
 {
  int i;
  public:
  alpha(int x)
  {
   cout<<"alpha constructor\n";
   i=x;
   }
   void display(){ cout<<"i="<<i<<endl;}
   };
```

```
class gamma
   {
    int j;
    alpha a;
    public:
    gamma(int c, int d):a(c)
    {
            cout<<"gamma constructor\n";
            j=d;
            }
            void display()
            {
             a.display();
             cout<<"j= "<<j<<endl;
             }
            };
```

```
int main()
        {
         gamma g(5,7);
         g.display();
         return 0;
         }
```

We can use as many as member objects as are required in a class

The constructor of member objects are called in the order in which they are declared in the nested class

```
#include<iostream>
 using namespace std;
 class A
 {
    public:
       class B
       {
          public:
             void display()
             {
                cout<<"Nested class"<<endl;
             }
       };
 };
 int main()
 {
    A::B in;
    in.display();
    return 0;
 }
```

H.W :- Create base class **Staff** with data members like name, code, qualification and experience. Create a derived class called **Teacher** and **Supporting_staff** from class **Staff.** Class **Teacher** that stores the number of subjects taught, name of subjects and specialization. Class **Supporting_staff** stores the professional skill and grade. Write parameterized constructors to initialize data members. Write display function to show the information of teacher and supporting staff. Write function to search teachers teaching specific subject. Also write function to search supporting staffs having specific professional skill. Write main to show usage of above functions. Create an array of objects. Also identify the type of inheritance.

```
class B {
public:
    int data;
    B(int d = 0) : data(d) {
        cout << "B::B(int): " << data<< endl;
    }
    ~B() {
        cout << "B::~B(): " << data << endl;
    }
};
```

```
class D: public B {
    int info;
public:
    // Explicit construction of Base
    D(int d, int i) : B(d), info(i) {
        cout << "D::D(int, int): " << data << ", " << info << endl;
    }

    // Default construction of Base
    D(int i) : info(i) {
        cout << "D::D(int): " << data << ", " << info << endl;
    }

    ~D() {
        cout << "D::~D(): " << data << ", " << info << endl;
    }
};
```

```
int main()
{
   B b(5);

   //  Explicit construction of Base
   D d1(1, 2);

  // Default construction of Base
  D d2(3);
return 0;
}
```

```
class B {
public:
    int data;
    B(int d = 0) : data(d) {
        cout << "B::B(int): " << data<< endl;
    }
    ~B() {
        cout << "B::~B(): " << data << endl;
    }
};
```

```
class D: public B {
    int info;
public:
    // Explicit construction of Base
    D(int d, int i) : B(d), info(i) {
        cout << "D::D(int, int): " << data << ", " << info << endl;
    }

    // Default construction of Base
    D(int i) : info(i) {
        cout << "D::D(int): " << data << ", " << info << endl;
    }

    ~D() {
        cout << "D::~D(): " << data << ", " << info << endl;
    }
};
```

```
int main()
{
   B b(5);

   //  Explicit construction of Base
   D d1(1, 2);

  // Default construction of Base
  //If base class constructor do not have default value →
Error

  D d2(3);
return 0;
}
```

```
class B {
public:
int data_;
B(int d = 0) : data_(d) {
    cout << "B::B(int): " << data_ << endl; }
~B() {
    cout << "B::~B(): " << data_ << endl; }
};
```

```
int main()
{
   B b(5);

   D d1(1, 2);

return 0;
}
```

```
class D: public B {
 int info_;
public:
D(int d, int i) {
B(d);
Info_= i ;
cout << "D::D(int, int): " << data_ << ", " << info_ << endl;
}

// Default construction of Base
D(int i) : info_(i) {
cout << "D::D(int): " << data_ << ", " << info_ << endl; }

~D() {
cout << "D::~D(): " << data_ << ", " << info_ << endl;
 }
};
```

```
class B {
public:
int data_;
B(int d = 0) : data_(d) {
    cout << "B::B(int): " << data_ << endl; }
~B() {
    cout << "B::~B(): " << data_ << endl; }
};
```

```
class D: public B {
 int info_;
public:
D(int d, int i) {
B(d);
Info_= i ;
cout << "D::D(int, int): " << data_ << ", " << info_ << endl; }

// Default construction of Base
D(int i) : info_(i) {
cout << "D::D(int): " << data_ << ", " << info_ << endl; }

~D() {
cout << "D::~D(): " << data_ << ", " << info_ << endl;
 }
};
```

```
int main()
{
   B b(5);


 D d1(1, 2);
```

Error: declaration of ‘B d’ shadows a parameter

/* as we must have base object prior to call to constructor of D. We haven’t supplied it so compiler call default constructor of B and set data_ =0 (default).

Now, code B(d) inside constructor try to create new object of B but as we already have an object of B created by compiler using default constructor of base class */

```
return 0;
}
```

```
#include<iostream>
#include<string>
using namespace std;

class Vehicle {
    int wheels_count;
    float max_speed;
    string manufacturer;
public:
    Vehicle(int, float, string);
    void vehicle_print();
};
Vehicle::Vehicle(int wheels_count, float max_speed, string manufacturer) {
    this->wheels_count = wheels_count;
    this->max_speed = max_speed;
    this->manufacturer = manufacturer;
}
void Vehicle::vehicle_print() {
    cout << "Wheel Count: " << wheels_count << endl;
    cout << "Max Speed: " << max_speed << endl;
    cout << "Manufacturer: " << manufacturer << endl;
}
```

```
int main() {

    Vehicle vehicle(2, 7.1, "Ambica carts");
    vehicle.vehicle_print();

    return 0;
}
```

Output ??

```
#include<iostream>
#include<string>
using namespace std;

class Vehicle {
    int wheels_count;
    float max_speed;
    string manufacturer;
public:
    Vehicle(int, float, string);
    void vehicle_print();
};
Vehicle::Vehicle(int wheels_count, float max_speed, string manufacturer) {
    this->wheels_count = wheels_count;
    this->max_speed = max_speed;
    this->manufacturer = manufacturer;
}
void Vehicle::vehicle_print() {
    cout << "Wheel Count: " << wheels_count << endl;
    cout << "Max Speed: " << max_speed << endl;
    cout << "Manufacturer: " << manufacturer << endl;
}
```

```
int main() {

    Vehicle vehicle(2, 7.1, "Ambica carts");
    vehicle.vehicle_print();

    return 0;
}
```

Wheel Count: 2
Max Speed: 7.1
Manufacturer: Ambica carts

```
class Vehicle {

    int wheels_count;
    float max_speed;
    string manufacturer;
public:
    Vehicle(int, float, string);
    void vehicle_print();
};

Vehicle::Vehicle(int wheels_count, float max_speed,
                                 string manufacturer):
wheels_count(wheels_count),
max_speed(max_speed),
manufacturer(manufacturer) {
    cout << "Vehicle constructor called\n";
}

void Vehicle::vehicle_print() {
    cout << "Wheel Count: " << wheels_count << endl;
    cout << "Max Speed: " << max_speed << endl;
    cout << "Manufacturer: " << manufacturer << endl;
}
```

```
class Automobile : public Vehicle{
    float engine_cc;
    int gear_count;
public:
Automobile(int wheels_count, float max_speed,
                            string manufacturer,
                            float engine_cc, int gear_count):
    engine_cc(engine_cc),
    gear_count(gear_count){}

    void print() {
        cout << "Wheel Count: " << wheels_count << endl;
        cout << "Max Speed: " << max_speed << endl;
        cout << "Manufacturer: " << manufacturer << endl;
        cout << "Engine Capacity: " << engine_cc << endl;
        cout << "Gear Count: " << gear_count << endl;
    }
};

int main() {
    Automobile automobile(2, 100, "Honda", 125, 4);
    return 0;
}
```

```
class Vehicle {

    int wheels_count;
    float max_speed;
    string manufacturer;
public:
    Vehicle(int, float, string);
    void vehicle_print();
};

Vehicle::Vehicle(int wheels_count, float max_speed,
                                    string manufacturer):
wheels_count(wheels_count),
max_speed(max_speed),
manufacturer(manufacturer) {
    cout << "Vehicle constructor called\n";
}

void Vehicle::vehicle_print() {
    cout << "Wheel Count: " << wheels_count << endl;
    cout << "Max Speed: " << max_speed << endl;
    cout << "Manufacturer: " << manufacturer << endl;
}
```

```
class Automobile : public Vehicle{
    float engine_cc;
    int gear_count;
public:
Automobile(int wheels_count, float max_speed,
                            string manufacturer,
                            float engine_cc, int gear_count):
    engine_cc(engine_cc),
    gear_count(gear_count){
// error: no matching function for call to 'Vehicle::Vehicle()'
}

    void print() {
        cout << "Wheel Count: " << wheels_count << endl;
        cout << "Max Speed: " << max_speed << endl;
        cout << "Manufacturer: " << manufacturer << endl;
        cout << "Engine Capacity: " << engine_cc << endl;
        cout << "Gear Count: " << gear_count << endl;
    }
};

int main() {
    Automobile automobile(2, 100, "Honda", 125, 4);
    return 0;}
```

```
class Vehicle {

    int wheels_count;
    float max_speed;
    string manufacturer;
public:
    Vehicle(int, float, string);
    void vehicle_print();
};

Vehicle::Vehicle(int wheels_count, float max_speed,
                                 string manufacturer):
wheels_count(wheels_count),
max_speed(max_speed),
manufacturer(manufacturer) {
    cout << "Vehicle constructor called\n";
}

void Vehicle::vehicle_print() {
    cout << "Wheel Count: " << wheels_count << endl;
    cout << "Max Speed: " << max_speed << endl;
    cout << "Manufacturer: " << manufacturer << endl;
}
```

```
class Automobile : public Vehicle{
    float engine_cc;
    int gear_count;
public:
Automobile(int wheels_count, float max_speed,
                        string manufacturer,
                        float engine_cc, int gear_count):
    engine_cc(engine_cc),
    gear_count(gear_count, Vehicle(wheels_count,
max_speed, manufacturer) {
        cout << "Automobile constructor called\n";
    }
    void print() {
        cout << "Wheel Count: " << wheels_count << endl;
        cout << "Max Speed: " << max_speed << endl;
        cout << "Manufacturer: " << manufacturer << endl;
        cout << "Engine Capacity: " << engine_cc << endl;
        cout << "Gear Count: " << gear_count << endl;
    }
};

int main() {
    Automobile automobile(2, 100, "Honda", 125, 4);
    return 0;}
```

```
class Vehicle {

    int wheels_count;
    float max_speed;
    string manufacturer;
public:
    Vehicle(int, float, string);
    void vehicle_print();
};
Vehicle::Vehicle(int wheels_count, float max_speed,
                                        string manufacturer):
wheels_count(wheels_count),
max_speed(max_speed),
manufacturer(manufacturer) {
    cout << "Vehicle constructor called\n";
}
void Vehicle::vehicle_print() {
    cout << "Wheel Count: " << wheels_count << endl;
    cout << "Max Speed: " << max_speed << endl;
    cout << "Manufacturer: " << manufacturer << endl;
}

class Automobile : public Vehicle{
    float engine_cc;
    int gear_count;
public:
    Automobile(int wheels_count, float max_speed,
                                        string manufacturer,
                float engine_cc, int gear_count):
    engine_cc(engine_cc),
    gear_count(gear_count),
    Vehicle(wheels_count, max_speed, manufacturer) {
        cout << "Automobile constructor called\n";
    }
    void print() {
        // error: 'int Vehicle::wheels_count' is private
        cout << "Wheel Count: " << wheels_count << endl;
        // similar error
        cout << "Max Speed: " << max_speed << endl;
        // similar error
        cout << "Manufacturer: " << manufacturer << endl;
        cout << "Engine Capacity: " << engine_cc << endl;
        cout << "Gear Count: " << gear_count << endl;
    }
};
int main() {
    Automobile automobile(2, 100, "Honda", 125, 4);
    automobile.print();
    return 0;
}
```

```
class Vehicle {
public:
    int wheels_count;
    float max_speed;
    string manufacturer;

    Vehicle(int, float, string);
    void vehicle_print();
};
Vehicle::Vehicle(int wheels_count, float max_speed,
                                        string manufacturer):
wheels_count(wheels_count),
max_speed(max_speed),
manufacturer(manufacturer) {
    cout << "Vehicle constructor called\n";
}
void Vehicle::vehicle_print() {
    cout << "Wheel Count: " << wheels_count << endl;
    cout << "Max Speed: " << max_speed << endl;
    cout << "Manufacturer: " << manufacturer << endl;
}
```

```
Vehicle constructor called
Automobile constructor called
Wheel Count: 2
Max Speed: 100
Manufacturer: Honda
Engine Capacity: 125
Gear Count: 4
```

```
class Automobile : public Vehicle{
    float engine_cc;
    int gear_count;
public:
Automobile(int wheels_count, float max_speed,
                                string manufacturer,
                float engine_cc, int gear_count):
    engine_cc(engine_cc),
    gear_count(gear_count),
    Vehicle(wheels_count, max_speed, manufacturer) {
        cout << "Automobile constructor called\n";
    }
    void print() {
        cout << "Wheel Count: " << wheels_count << endl;
        cout << "Max Speed: " << max_speed << endl;
        cout << "Manufacturer: " << manufacturer << endl;
        cout << "Engine Capacity: " << engine_cc << endl;
        cout << "Gear Count: " << gear_count << endl;
    }
};
int main() {
    Automobile automobile(2, 100, "Honda", 125, 4);
    automobile.print();
    return 0;
}
```

```
class Vehicle {

    int wheels_count;
    float max_speed;
    string manufacturer;
public:
    Vehicle(int, float, string);
    void vehicle_print();
};
Vehicle::Vehicle(int wheels_count, float max_speed,
                                        string manufacturer):
wheels_count(wheels_count),
max_speed(max_speed),
manufacturer(manufacturer) {
    cout << "Vehicle constructor called\n";
}
void Vehicle::vehicle_print() {
    cout << "Wheel Count: " << wheels_count << endl;
    cout << "Max Speed: " << max_speed << endl;
    cout << "Manufacturer: " << manufacturer << endl;
}
```

```
Vehicle constructor called
Automobile constructor called
Wheel Count: 2
Max Speed: 100
Manufacturer: Honda
Engine Capacity: 125
Gear Count: 4
```

```
class Automobile : public Vehicle{
    float engine_cc;
    int gear_count;
public:
Automobile(int wheels_count, float max_speed,
                        string manufacturer,
            float engine_cc, int gear_count):
    engine_cc(engine_cc),
    gear_count(gear_count),
    Vehicle(wheels_count, max_speed,
manufacturer) {
        cout << "Automobile constructor called\n";
    }
    void print() {
            vehicle_print();
        cout << "Engine Capacity: " << engine_cc <<
endl;
        cout << "Gear Count: " << gear_count << endl;
    }
};
int main() {
    Automobile automobile(2, 100, "Honda", 125, 4);
    automobile.print();
    return 0;}
```

```
class Vehicle {
protected:
    int wheels_count;
    float max_speed;
    string manufacturer;

    Vehicle(int, float, string);
    void vehicle_print();
};
Vehicle::Vehicle(int wheels_count, float max_speed,
                                        string manufacturer):
wheels_count(wheels_count),
max_speed(max_speed),
manufacturer(manufacturer) {
    cout << "Vehicle constructor called\n";
}
void Vehicle::vehicle_print() {
    cout << "Wheel Count: " << wheels_count << endl;
    cout << "Max Speed: " << max_speed << endl;
    cout << "Manufacturer: " << manufacturer << endl;
}
```

```
Vehicle constructor called
Automobile constructor called
Wheel Count: 2
Max Speed: 100
Manufacturer: Honda
Engine Capacity: 125
Gear Count: 4
```

```
class Automobile : public Vehicle{
    float engine_cc;
    int gear_count;
public:
 Automobile(int wheels_count, float max_speed,
                                string manufacturer,
                float engine_cc, int gear_count):
    engine_cc(engine_cc),
    gear_count(gear_count),
    Vehicle(wheels_count, max_speed, manufacturer) {
        cout << "Automobile constructor called\n";
    }
    void print() {
        cout << "Wheel Count: " << wheels_count << endl;
        cout << "Max Speed: " << max_speed << endl;
        cout << "Manufacturer: " << manufacturer << endl;
        cout << "Engine Capacity: " << engine_cc << endl;
        cout << "Gear Count: " << gear_count << endl;
    }
};
int main() {
    Automobile automobile(2, 100, "Honda", 125, 4);
    automobile.print();
    return 0;
}
```

```
class Vehicle {
protected:
    int wheels_count;
    float max_speed;
    string manufacturer;

    Vehicle(int, float, string);
    void vehicle_print();
};
Vehicle::Vehicle(int wheels_count, float max_speed,
                                                string manufacturer):
wheels_count(wheels_count),
max_speed(max_speed),
manufacturer(manufacturer) {
    cout << "Vehicle constructor called\n";
}
void Vehicle::vehicle_print() {
    cout << "Wheel Count: " << wheels_count << endl;
    cout << "Max Speed: " << max_speed << endl;
    cout << "Manufacturer: " << manufacturer << endl;
}
```

```
class Automobile : public Vehicle{
    float engine_cc;
    int gear_count;
public:
Automobile(int wheels_count, float max_speed,
                                string manufacturer,
                float engine_cc, int gear_count):
    engine_cc(engine_cc),
    gear_count(gear_count),
    Vehicle(wheels_count, max_speed, manufacturer) {
        cout << "Automobile constructor called\n";
    }
    void print() {
        cout << "Wheel Count: " << wheels_count << endl;
        cout << "Max Speed: " << max_speed << endl;
        cout << "Manufacturer: " << manufacturer << endl;
        cout << "Engine Capacity: " << engine_cc << endl;
        cout << "Gear Count: " << gear_count << endl;
    }
};
int main() {
    Automobile automobile(2, 100, "Honda", 125, 4);
    automobile.vehicle_print();
    return 0;
}
```

```
class Vehicle {
protected:
    int wheels_count;
    float max_speed;
    string manufacturer;

    Vehicle(int, float, string);
    void vehicle_print();
};
Vehicle::Vehicle(int wheels_count, float max_speed,
                                        string manufacturer):
wheels_count(wheels_count),
max_speed(max_speed),
manufacturer(manufacturer) {
    cout << "Vehicle constructor called\n";
}
void Vehicle::vehicle_print() {
    cout << "Wheel Count: " << wheels_count << endl;
    cout << "Max Speed: " << max_speed << endl;
    cout << "Manufacturer: " << manufacturer << endl;
}
```

```
class Automobile : public Vehicle{
    float engine_cc;
    int gear_count;
public:
Automobile(int wheels_count, float max_speed,
                                string manufacturer,
                float engine_cc, int gear_count):
    engine_cc(engine_cc),
    gear_count(gear_count),
    Vehicle(wheels_count, max_speed, manufacturer) {
        cout << "Automobile constructor called\n";
    }
    void print() {
        cout << "Wheel Count: " << wheels_count << endl;
        cout << "Max Speed: " << max_speed << endl;
        cout << "Manufacturer: " << manufacturer << endl;
        cout << "Engine Capacity: " << engine_cc << endl;
        cout << "Gear Count: " << gear_count << endl;
    }
};
int main() {
    Automobile automobile(2, 100, "Honda", 125, 4);
    // error: 'void Vehicle::vehicle_print()' is protected
    automobile.vehicle_print();
    return 0;
}
```

```
class Vehicle {
protected:
    int wheels_count;
    float max_speed;
    string manufacturer;

    Vehicle(int, float, string);
    void vehicle_print();
};
Vehicle::Vehicle(int wheels_count, float max_speed,
                                        string manufacturer):
wheels_count(wheels_count),
max_speed(max_speed),
manufacturer(manufacturer) {
    cout << "Vehicle constructor called\n";
}
void Vehicle::vehicle_print() {
    cout << "Wheel Count: " << wheels_count << endl;
    cout << "Max Speed: " << max_speed << endl;
    cout << "Manufacturer: " << manufacturer << endl;
}
```

```
class Automobile : public Vehicle{
    float engine_cc;
    int gear_count;
public:
Automobile(int wheels_count, float max_speed,
                                string manufacturer,
                float engine_cc, int gear_count):
    engine_cc(engine_cc),
    gear_count(gear_count),
    Vehicle(wheels_count, max_speed, manufacturer) {
        cout << "Automobile constructor called\n";
    }
    void print() {
        vehicle_print();
        cout << "Engine Capacity: " << engine_cc << endl;
        cout << "Gear Count: " << gear_count << endl;
    }
};
int main() {
    Automobile automobile(2, 100, "Honda", 125, 4);
    automobile.print();
    return 0;
}
```

Access Specifier	Access from methods of class itself	Access from methods of derived class	Acess from methods of other classes and independent functions
public	Allowed	Allowed	Allowed
protected	Allowed	Allowed	Not Allowed
private	Allowed	Not Allowed	Not Allowed

```
class Vehicle {
   int wheels_count;
protected:
   float max_speed;
public:
   string manufacturer;
   Vehicle(int, float, string);
   void vehicle_print();
};
Vehicle::Vehicle(int wheels_count, float
max_speed,
string manufacturer):
wheels_count(wheels_count),
max_speed(max_speed),
manufacturer(manufacturer){
   cout << "Vehicle constructor called\n";
}
void Vehicle::vehicle_print() {
   cout << "Wheel Count: " << wheels_count <<
endl;
   cout << "Max Speed: " << max_speed << endl;
   cout << "Manufacturer: " << manufacturer <<
endl;
}
```

```
class Automobile : public Vehicle{
   float engine_cc;
   int gear_count;
public:
Automobile(int wheels_count, float max_speed,
                              string manufacturer,
             float engine_cc, int gear_count):
   engine_cc(engine_cc),
   gear_count(gear_count),
   Vehicle(wheels_count, max_speed, manufacturer) {
      cout << "Automobile constructor called\n";
   }
   void print() {
      cout << "Wheel Count: " << wheels_count << endl;
      cout << "Max Speed: " << max_speed << endl;
      cout << "Manufacturer: " << manufacturer << endl;
      cout << "Engine Capacity: " << engine_cc << endl;
      cout << "Gear Count: " << gear_count << endl;
   }
};
int main() {
   Automobile automobile(2, 100, "Honda", 125, 4);
   cout << "Wheel Count: " << automobile.wheels_count <<
endl;
   cout << "Max Speed: " << automobile.max_speed << endl;
   cout << "Manufacturer: " << automobile.manufacturer <<
endl;
   automobile.print();
   return 0;
}
```

```
class Vehicle {
    int wheels_count;
protected:
    float max_speed;
public:
    string manufacturer;
    Vehicle(int, float, string);
    void vehicle_print();
};
Vehicle::Vehicle(int wheels_count, float max_speed,
string manufacturer):
wheels_count(wheels_count),
max_speed(max_speed),
manufacturer(manufacturer){
    cout << "Vehicle constructor called\n";
}
void Vehicle::vehicle_print() {
    cout << "Wheel Count: " << wheels_count << endl;
    cout << "Max Speed: " << max_speed << endl;
    cout << "Manufacturer: " << manufacturer << endl;
}

class Automobile : public Vehicle{
    float engine_cc;
    int gear_count;
public:
```

```
    Automobile(int wheels_count, float max_speed,
                                string manufacturer,
                float engine_cc, int gear_count):
    engine_cc(engine_cc),
    gear_count(gear_count),
    Vehicle(wheels_count, max_speed, manufacturer) {
        cout << "Automobile constructor called\n";
    }
    void print() {
        // error: 'int Vehicle::wheels_count' is private
        cout << "Wheel Count: " << wheels_count << endl;
        cout << "Max Speed: " << max_speed << endl;
        cout << "Manufacturer: " << manufacturer << endl;
        cout << "Engine Capacity: " << engine_cc << endl;
        cout << "Gear Count: " << gear_count << endl;
    }
};
int main() {
    Automobile automobile(2, 100, "Honda", 125, 4);
    // error: 'int Vehicle::wheels_count' is private
    cout << "Wheel Count: " << automobile.wheels_count << endl;
    // error: 'float Vehicle::max_speed' is protected
    cout << "Max Speed: " << automobile.max_speed << endl;
    cout << "Manufacturer: " << automobile.manufacturer << endl;
    automobile.print();
    return 0;
}
```

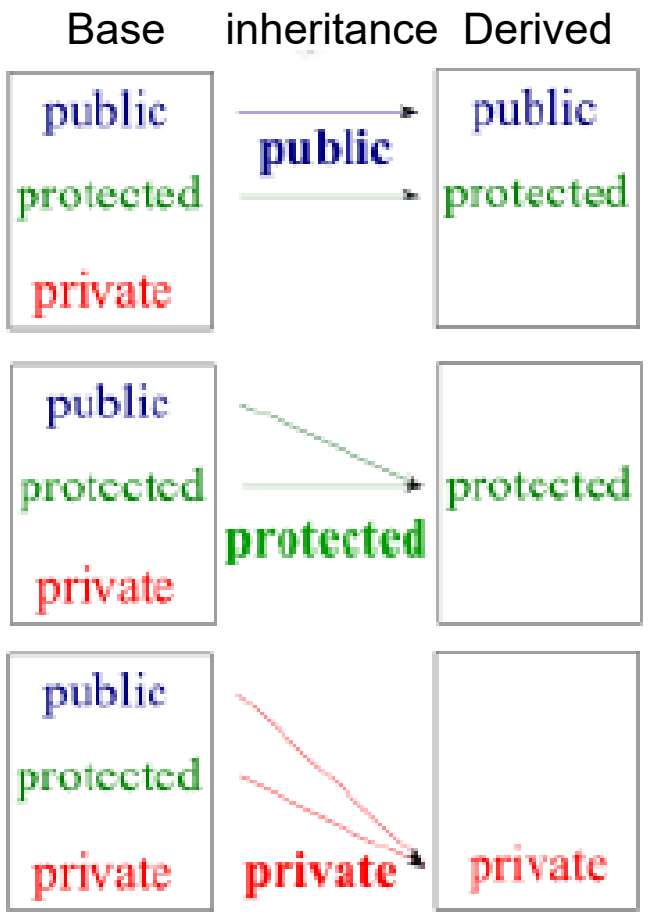
```
class Vehicle {
    int wheels_count;
protected:
    float max_speed;
public:
    string manufacturer;
    Vehicle(int, float, string);
    void vehicle_print();
};
Vehicle::Vehicle(int wheels_count, float max_speed,
string manufacturer):
wheels_count(wheels_count),
max_speed(max_speed),
manufacturer(manufacturer){
    cout << "Vehicle constructor called\n";
}
void Vehicle::vehicle_print() {
    cout << "Wheel Count: " << wheels_count << endl;
    cout << "Max Speed: " << max_speed << endl;
    cout << "Manufacturer: " << manufacturer << endl;
}

class Automobile : public Vehicle{
    float engine_cc;
    int gear_count;
public:
```

```
Vehicle constructor called
Automobile constructor called
Manufacturer: Honda //(from main)
Max Speed: 100
Manufacturer: Honda
Engine Capacity: 125
Gear Count: 4
```

```
    Automobile(int wheels_count, float max_speed,
                                string manufacturer,
            float engine_cc, int gear_count):
    engine_cc(engine_cc),
    gear_count(gear_count),
    Vehicle(wheels_count, max_speed, manufacturer) {
        cout << "Automobile constructor called\n";
    }
    void print() {
        // error: 'int Vehicle::wheels_count' is private
        // cout << "Wheel Count: " << wheels_count << endl;
        cout << "Max Speed: " << max_speed << endl;
        cout << "Manufacturer: " << manufacturer << endl;
        cout << "Engine Capacity: " << engine_cc << endl;
        cout << "Gear Count: " << gear_count << endl;
    }
};
int main() {
    Automobile automobile(2, 100, "Honda", 125, 4);
    // error: 'int Vehicle::wheels_count' is private
    // cout << "Wheel Count: " << automobile.wheels_count << endl;
    // error: 'float Vehicle::max_speed' is protected
    // cout << "Max Speed: " << automobile.max_speed << endl;
    cout << "Manufacturer: " << automobile.manufacturer << endl;
    automobile.print();
    return 0;
}
```

class derived-class : **visibility-mode** base-class

Visibility mode	Base class	Derived class
public	public	public
	protected	protected
	private	**No access**
protected	public	protected
	protected	protected
	private	**No access**
private	public	private
	protected	private
	private	**No access**



```
class Vehicle {
    int wheels_count;
protected:
    float max_speed;
public:
    string manufacturer;
    Vehicle(int, float, string);
    void vehicle_print();
};
Vehicle::Vehicle(int wheels_count, float max_speed,
string manufacturer):
wheels_count(wheels_count),
max_speed(max_speed),
manufacturer(manufacturer){
    cout << "Vehicle constructor called\n";
}
void Vehicle::vehicle_print() {
    cout << "Wheel Count: " << wheels_count << endl;
    cout << "Max Speed: " << max_speed << endl;
    cout << "Manufacturer: " << manufacturer << endl;
}

class Automobile : protected Vehicle{
    float engine_cc;
    int gear_count;
public:
```

```
Vehicle constructor called
Automobile constructor called
Max Speed: 100
Manufacturer: Honda
Engine Capacity: 125
Gear Count: 4
```

```
    Automobile(int wheels_count, float max_speed,
                                string manufacturer,
            float engine_cc, int gear_count):
    engine_cc(engine_cc),
    gear_count(gear_count),
    Vehicle(wheels_count, max_speed, manufacturer) {
        cout << "Automobile constructor called\n";
    }
    void print() {
        // error: ‘int Vehicle::wheels_count’ is private
        // cout << "Wheel Count: " << wheels_count << endl;
        cout << "Max Speed: " << max_speed << endl;
        cout << "Manufacturer: " << manufacturer << endl;
        cout << "Engine Capacity: " << engine_cc << endl;
        cout << "Gear Count: " << gear_count << endl;
    }
};
int main() {
    Automobile automobile(2, 100, "Honda", 125, 4);
    // error: ‘int Vehicle::wheels_count’ is private
    // cout << "Wheel Count: " << automobile.wheels_count << endl;
    // error: ‘float Vehicle::max_speed’ is protected
    // cout << "Max Speed: " << automobile.max_speed << endl;
    // error: string Vehicle::manufacturer is inaccessible
    // cout << "Manufacturer: " << automobile.manufacturer << endl;
    automobile.print();
    return 0;
}
```

Protected Understanding:
For Derive class it is like public
For Other classes and functions it is like private

```
class Vehicle {
    int wheels_count;
protected:
    float max_speed;
public:
    string manufacturer;
    Vehicle(int, float, string);
    void vehicle_print();
};
Vehicle::Vehicle(int wheels_count, float max_speed,
string manufacturer):
wheels_count(wheels_count),
max_speed(max_speed),
manufacturer(manufacturer){
    cout << "Vehicle constructor called\n";
}
void Vehicle::vehicle_print() {
    cout << "Wheel Count: " << wheels_count << endl;
    cout << "Max Speed: " << max_speed << endl;
    cout << "Manufacturer: " << manufacturer << endl;
}

class Automobile : private Vehicle{
    float engine_cc;
    int gear_count;
public:
```

```
Vehicle constructor called
Automobile constructor called
Max Speed: 100
Manufacturer: Honda
Engine Capacity: 125
Gear Count: 4
```

```
    Automobile(int wheels_count, float max_speed,
                                string manufacturer,
            float engine_cc, int gear_count):
    engine_cc(engine_cc),
    gear_count(gear_count),
    Vehicle(wheels_count, max_speed, manufacturer) {
        cout << "Automobile constructor called\n";
    }
    void print() {
        // error: ‘int Vehicle::wheels_count’ is private
        // cout << "Wheel Count: " << wheels_count << endl;
        cout << "Max Speed: " << max_speed << endl;
        cout << "Manufacturer: " << manufacturer << endl;
        cout << "Engine Capacity: " << engine_cc << endl;
        cout << "Gear Count: " << gear_count << endl;
    }
};
int main() {
    Automobile automobile(2, 100, "Honda", 125, 4);
    // error: ‘int Vehicle::wheels_count’ is private
    // cout << "Wheel Count: " << automobile.wheels_count << endl;
    // error: ‘float Vehicle::max_speed’ is protected
    // cout << "Max Speed: " << automobile.max_speed << endl;
    // error: string Vehicle::manufacturer is inaccessible
    // cout << "Manufacturer: " << automobile.manufacturer << endl;
    automobile.print();
    return 0;
}
```